

MAC Flooding Attack

Objetivo del Laboratorio

Demostrar el ataque MAC Flooding en un entorno de laboratorio controlado usando EVE-NG, evidenciando cómo un atacante puede saturar la tabla CAM de un switch Cisco para convertirlo en un HUB y capturar tráfico que no le pertenece.

Objetivo del Script

Inundar la tabla CAM (Content Addressable Memory) del switch SW1 con miles de direcciones MAC falsas hasta agotarla, forzando al switch a realizar flooding de todos los paquetes hacia todos los puertos.

Parámetros

Parámetro	Valor	Descripción
Interfaz	eth0	Interfaz de Kali hacia SW1
Paquetes	10000	Cantidad de MACs falsas
Red objetivo	172.25.78.0/24	Red del laboratorio

Requisitos para utilizar la herramienta

- Kali Linux con Python 3 instalado
- Librería Scapy instalada (pip install scapy)
- Permisos root (sudo)
- Conectividad capa 2 con el switch objetivo

Funcionamiento del Script

Generación de MACs aleatorias

La función `random_mac()` genera direcciones MAC completamente aleatorias de 6 bytes para cada paquete enviado.

Generación de IPs aleatorias

La función `random_ip()` genera IPs dentro del rango 172.25.78.1-254 para simular hosts ficticios.

Construcción del paquete

Cada paquete es construido con Scapy usando capas Ethernet y ARP con MACs e IPs falsas tanto en origen como en destino.

Envío masivo

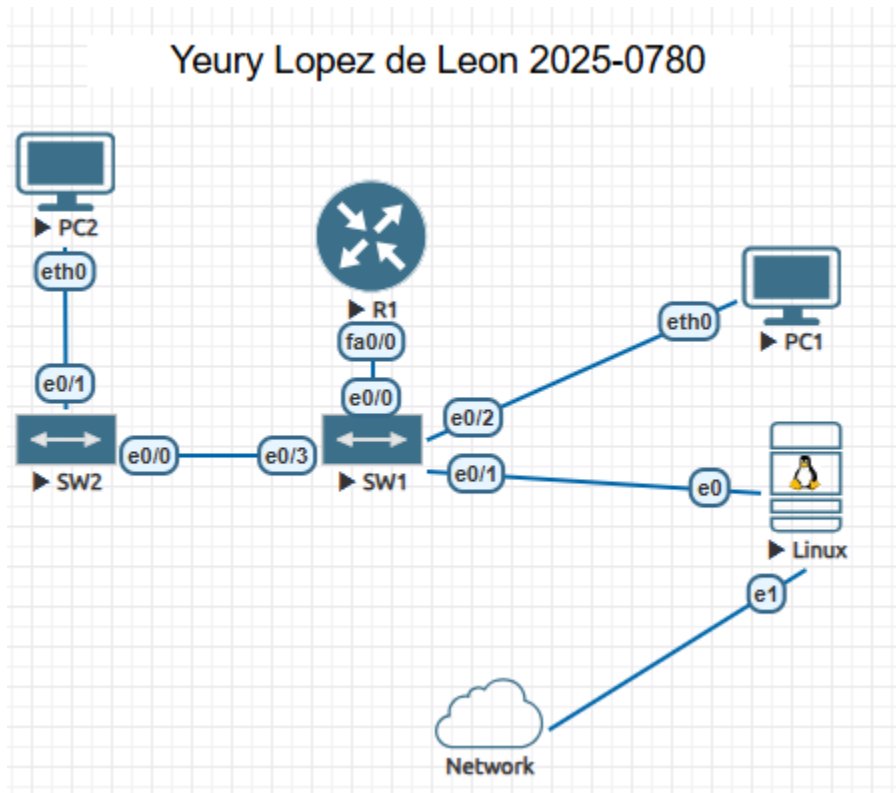
Se envían 10,000 paquetes por la interfaz `eth0` sin esperar respuesta (`sendp` con `verbose=False`).

Progreso

El script muestra el progreso cada 100 paquetes enviados con timestamp.

Documentación de la Red

Topología



Direccionamiento IP

Dispositivo	Interfaz	IP	RoI
R1	fa0/0	172.25.78.1/24	Gateway + DHCP
SW1	VLAN1	172.25.78.2/24	Switch Core - Root Bridge
SW2	VLAN1	172.25.78.3/24	Switch Acceso
Kali	eth0	172.25.78.10/24	Atacante
PC1	eth0	172.25.78.20/24	Víctima 1 (estática)
PC2	eth0	172.25.78.21/24	Víctima 2 (DHCP)

Conexiones

Dispositivo A	Interfaz	Dispositivo B	Interfaz	Tipo
R1	fa0/0	SW1	e0/0	Ethernet
SW1	e0/1	Kali	eth0	Ethernet
SW1	e0/2	PC1	eth0	Ethernet
SW1	e0/3	SW2	e0/0	Ethernet
SW2	e0/1	PC2	eth0	Ethernet

Herramientas Utilizadas

- EVE-NG Community Edition
- Cisco IOL L2 v15.1 (SW1, SW2)
- Cisco IOS 3725 v12.4 Dynamips (R1)
- Kali Linux 2024
- Python 3 + Scapy
- VPCS (PC1, PC2)

Evidencias

Tabla CAM antes del ataque

```
SW1>show mac address-table count

Mac Entries for Vlan 1:
-----
Dynamic Address Count   : 2
Static Address Count    : 0
Total Mac Addresses     : 2

Total Mac Address Space Available: 183202564

SW1>show mac address-table
      Mac Address Table
-----
Vlan    Mac Address      Type        Ports
----    -
1       0050.0000.0600    DYNAMIC     Et0/1
1       c201.0d42.0000    DYNAMIC     Et0/0
Total Mac Addresses for this criterion: 2
```

Ejecución del script

```
[sudo] password for kali:
=====
MAC FLOODING ATTACK
Autor   : Yeury Lopez
Matricula: 2025-0780
=====

[*] Interfaz objetivo : eth0
[*] Paquetes a enviar : 10000
[*] Iniciando ataque  : 2026-05-31 17:08:08
=====

[+] Paquetes enviados: 100/10000
[+] Paquetes enviados: 200/10000
[+] Paquetes enviados: 300/10000
[+] Paquetes enviados: 400/10000
[+] Paquetes enviados: 500/10000
[+] Paquetes enviados: 600/10000
[+] Paquetes enviados: 700/10000
```

Tabla CAM llena durante el ataque

```
SW1>show mac address-table count

Mac Entries for Vlan 1:
-----
Dynamic Address Count   : 716
Static Address Count    : 0
Total Mac Addresses     : 716

Total Mac Address Space Available: 183202564

SW1>show mac address-table count
```

Tráfico capturado en Kali

```
kali@kali: ~  
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes  
17:10:13.367522 ARP, Request who-has 172.25.78.206 (0c:f2:14:34:4c:42) tell 172.25.78.36, length 28  
17:10:13.399847 ARP, Request who-has 172.25.78.187 (11:8f:ec:28:19:88) tell 172.25.78.76, length 28  
17:10:13.444883 ARP, Request who-has 172.25.78.33 (dd:a9:1c:b1:f0:cf) tell 172.25.78.179, length 28  
17:10:13.478255 ARP, Request who-has 172.25.78.226 (4e:99:52:49:66:bf) tell 172.25.78.218, length 28  
17:10:13.511831 ARP, Request who-has 172.25.78.242 (c8:f9:43:c7:8c:61) tell 172.25.78.66, length 28  
17:10:13.535221 ARP, Request who-has 172.25.78.213 (ff:ff:e5:5a:12:74) tell 172.25.78.223, length 28  
17:10:13.552480 ARP, Request who-has 172.25.78.166 (ef:34:1e:81:fe:c2) tell 172.25.78.43, length 28  
17:10:13.575621 ARP, Request who-has 172.25.78.233 (16:f6:b2:01:f7:5b) tell 172.25.78.247, length 28  
17:10:13.599459 ARP, Request who-has 172.25.78.123 (98:a5:44:60:ff:8d) tell 172.25.78.86, length 28  
17:10:13.623002 ARP, Request who-has 172.25.78.150 (c4:16:02:6f:e2:9f) tell 172.25.78.105, length 28  
17:10:13.655484 ARP, Request who-has 172.25.78.1 (c7:c6:6a:4b:6b:9e) tell 172.25.78.213, length 28  
17:10:13.699735 ARP, Request who-has 172.25.78.227 (b2:5d:0d:77:4f:ee) tell 172.25.78.193, length 28  
17:10:13.719638 ARP, Request who-has 172.25.78.74 (25:50:57:f7:10:0b) tell 172.25.78.14, length 28  
17:10:13.743237 ARP, Request who-has 172.25.78.208 (8b:37:5b:87:76:c9) tell 172.25.78.13, length 28  
17:10:13.767669 ARP, Request who-has 172.25.78.219 (cb:87:1b:6c:c6:e6) tell 172.25.78.145, length 28  
17:10:13.811041 ARP, Request who-has 172.25.78.205 (93:33:75:e9:90:d7) tell 172.25.78.10, length 28  
17:10:13.867707 ARP, Request who-has 172.25.78.36 (61:10:ca:ee:cc:46) tell 172.25.78.162, length 28  
17:10:13.885006 ARP, Request who-has 172.25.78.254 (10:b9:40:59:ec:f0) tell 172.25.78.137, length 28  
17:10:13.941879 ARP, Request who-has 172.25.78.245 (49:4e:b2:21:b2:03) tell 172.25.78.4, length 28  
17:10:13.987644 ARP, Request who-has 172.25.78.142 (f9:45:b0:fe:de:19) tell 172.25.78.244, length 28  
17:10:14.037004 ARP, Request who-has 172.25.78.170 (0e:6a:68:57:05:83) tell 172.25.78.167, length 28  
17:10:14.071658 ARP, Request who-has 172.25.78.50 (cb:7f:d8:41:8f:9a) tell 172.25.78.9, length 28  
17:10:14.104694 ARP, Request who-has 172.25.78.136 (0a:35:d6:ad:2f:24) tell 172.25.78.245, length 28  
17:10:14.153215 ARP, Request who-has 172.25.78.96 (ad:1a:34:3d:e4:cf) tell 172.25.78.26, length 28  
17:10:14.204441 ARP, Request who-has 172.25.78.211 (79:e9:e5:d4:02:b9) tell 172.25.78.46, length 28  
17:10:14.223109 ARP, Request who-has 172.25.78.111 (b3:29:79:2e:c9:b4) tell 172.25.78.69, length 28
```

Contramedidas

Port Security en SW1

Configurar Port Security limita la cantidad de MACs permitidas por puerto, bloqueando el ataque:

Verificación de la contramedida

```
SW1#show port-security interface ethernet 0/1
Port Security          : Enabled
Port Status            : Secure-up
Violation Mode         : Restrict
Aging Time             : 2 mins
Aging Type             : Absolute
SecureStatic Address Aging : Disabled
Maximum MAC Addresses  : 5
Total MAC Addresses    : 5
Configured MAC Addresses : 0
Sticky MAC Addresses   : 0
Last Source Address:Vlan : 021e.44dd.f8bf:1
Security Violation Count : 301

SW1#
*May 31 21:13:20.699: %PORT_SECURITY-3-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 0c31.383b.421e on port Ethernet0/1.
SW1#
*May 31 21:13:25.802: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by MAC address 8a5f.f035.7f60 on port Ethernet0/1.
SW1#
```

Con Port Security activo, el switch bloquea automáticamente cualquier intento de registrar más de 5 MACs por puerto, registrando las violaciones sin afectar el tráfico legítimo.

Video: <https://youtu.be/s0NAiirPzzI>

Github: https://github.com/ylopez03x/YeuryLopez_2025-0780_MAC-Flooding